

Frontend Interview Handbook — Part 1: HTML, CSS, JavaScript

For: Samuel Obeng · Frontend (full-stack background) role prep, ViewEngine and general senior frontend interviews

SECTION A: HTML & SEMANTIC HTML

Q1: What is semantic HTML, and why does it matter beyond just "best practice"?

Answer: Semantic HTML means using elements that describe their meaning/content, not just their appearance — `<nav>`, `<article>`, `<header>`, `<button>` instead of generic `<div>`s styled to look like those things. A `<div onclick="">` might look like a button, but it carries no meaning to a screen reader, a search crawler, or an AI system parsing the page.

Why the interviewer asks this: It's a filter question — separates people who treat HTML as "just markup I don't think about" from people who understand it's the foundation both accessibility and machine-readability sit on. At a company like ViewEngine, this is a load-bearing question, not a throwaway one.

Common follow-ups:

- "Give me 5 semantic elements and when you'd use each."
- "What breaks if you build an entire page out of `<div>`s?"

Best practice: Default to the most specific semantic element available (`<button>` over `<div role="button">`) — you get free keyboard interaction, focus handling, and screen reader behavior instead of having to reimplement it with ARIA.

Real-world example: On Rentmann Properties, using `<article>` for each listing card and a proper `<h2>/<h3>` hierarchy instead of styled divs would mean a screen reader user (and a crawler) understands "this is a discrete unit of content" without extra work.

Common mistakes: Using `<div>/<span>` for everything and patching behavior in with JS and ARIA; using multiple `<h1>` tags per page; skipping heading levels (h2 straight to h4).

html

```
<!-- Bad -->
<div class="header">...</div>
<div class="nav">...</div>
<div class="title">Article Title</div>

<!-- Good -->
<header>...</header>
<nav>...</nav>
<h1>Article Title</h1>
```

Q2: What's the difference between `<section>`, `<article>`, and `<div>`?

Answer: `<article>` is for self-contained content that would make sense distributed/syndicated on its own (a blog post, a product card, a comment). `<section>` groups related content that's usually part of a larger whole and typically has its own heading (a chapter, a themed part of a page). `<div>` is a generic container with zero semantic meaning — use it only when no semantic element fits, purely for styling/layout hooks.

Why asked: Tests whether you actually understand the semantic distinctions or just know the element names exist.

Common follow-up: "Would you put a `<section>` inside an `<article>`, or the reverse — or both?" (Answer: both are valid depending on structure — an `<article>` can contain `<section>`s, e.g., a long blog post split into named sections; a `<section>` can also contain multiple `<article>`s, e.g., a "latest posts" section listing several post summaries.)

Common mistake: Using `<section>` purely as a styling wrapper — if it has no heading and isn't a meaningful thematic grouping, it should probably be a `<div>`.

Q3: How does the browser parse HTML into the DOM, at a high level?

Answer: The browser reads HTML top to bottom, tokenizing it and building a tree of nodes (the DOM). When it hits a `<script>` tag without `async`/`defer`, parsing pauses until the script downloads and executes (render-blocking), then resumes. CSS is parsed in parallel into the CSSOM. Once both DOM and CSSOM are ready, the browser combines them into the **render tree**, then does **layout** (compute geometry) and **paint** (draw pixels).

Why asked: Tests whether you understand *why* script placement and `async`/`defer` matter — directly relevant to performance/Core Web Vitals questions.

Follow-up: "Where should you put `<script>` tags, and what's the difference between `async` and `defer`?" **Answer:** Prefer end of `<body>`, or `<head>` with `defer`. `async` downloads in parallel and executes as soon as it's ready (order not guaranteed, can

interrupt parsing) — good for independent scripts like analytics. `defer` downloads in parallel but executes only after parsing completes, in document order — good for scripts that depend on the DOM or on each other.

Q4: What is `alt` text for, and what makes it "good"?

Answer: `alt` text describes an image's content/purpose for screen reader users and for cases where the image fails to load; it's also a signal search engines use to understand image content. Good alt text is concise and describes the *purpose* the image serves in context — not "image of a house" but "3-bedroom colonial house exterior, front view" if it's a real estate listing photo. Purely decorative images should have `alt=""` (empty, not omitted) so screen readers skip them rather than reading the filename.

Common mistake: Keyword-stuffing alt text for SEO ("house for sale cheap house buy house Ghana") — this is both bad practice and actively penalized; it hurts accessibility and looks spammy to modern ranking/AI systems that weigh naturalness of content.

SECTION B: CSS & RESPONSIVE DESIGN

Q5: Explain the CSS box model.

Answer: Every element is a box made of, from inside out: **content, padding, border, margin**. By default (`box-sizing: content-box`), `width`/`height` apply only to the content area, so padding and border add to the total rendered size. `box-sizing: border-box` makes `width`/`height` include padding and border, which is far more predictable — most modern resets set `border-box` globally.

CSS

```
* { box-sizing: border-box; }
```

Why asked: Foundational, but people who haven't internalized it consistently misjudge layout sizes — instant signal of experience level.

Q6: Flexbox vs Grid — when do you use each?

Answer: Flexbox is one-dimensional — great for laying out items in a row or column (nav bars, button groups, centering content). Grid is two-dimensional — great when you need to control rows *and* columns simultaneously (page layouts, card grids, dashboards). In practice they're often combined: Grid for the overall page structure, Flexbox for alignment within individual components.

Follow-up: "How would you build a responsive card grid that reflows from 4 columns to 1 as the viewport shrinks, without media queries?"

CSS

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 1rem;  
}
```

This is a strong answer because it shows you know `auto-fit`/`minmax` — a common "do you actually know Grid" filter.

Q7: What's the difference between `em`, `rem`, `%`, `vw/vh`, and `px`?

Answer: `px` is an absolute unit. `em` is relative to the *parent's* (or element's own) font size, which can compound unpredictably in nested components. `rem` is relative to the root (`<html>`) font size — predictable, which is why it's preferred for most spacing/typography in scalable design systems. `%` is relative to the parent container's corresponding dimension. `vw`/`vh` are relative to the viewport width/height — useful for full-bleed layout but risky for font sizing alone (can become unreadable at extreme viewport sizes; `clamp()` is often better).

CSS

```
font-size: clamp(1rem, 2vw + 0.5rem, 1.5rem);
```

Q8: How do you approach responsive design — mobile-first or desktop-first, and why?

Answer: Mobile-first is generally preferred: write base styles for small screens, then use `min-width` media queries to progressively enhance for larger viewports. This tends to produce leaner CSS (mobile constraints force simpler layouts by default) and aligns with how Google indexes — mobile-first indexing means Google predominantly uses the mobile version of a page for ranking, so a frontend engineer at an SEO-focused company should default to this mindset.

CSS

```
/* mobile-first base */  
.container { padding: 1rem; }  
  
@media (min-width: 768px) {  
  .container { padding: 2rem; }  
}
```

Common mistake: Writing desktop-first with `max-width` queries, which often leads to bloated overrides and is a worse fit for how modern SEO/mobile indexing actually evaluates pages.

Q9: What causes layout shift (CLS), and how do you prevent it in CSS?

Answer: Layout shift happens when content that's already visible moves because something loads/resizes after initial paint — images without dimensions, web fonts swapping in with different metrics (FOIT/FOUT), ads or embeds injecting late, dynamically injected banners above existing content. Prevention: always set `width`/`height` or `aspect-ratio` on images and video; use `font-display: swap` carefully and consider preloading critical fonts; reserve space for dynamic content with min-height placeholders.

css

```
img { aspect-ratio: 16 / 9; width: 100%; height: auto; }
```

(Covered in more depth in Part 5 alongside full Core Web Vitals.)

Q10: What's the difference between `position: relative`, `absolute`, `fixed`, and `sticky`?

Answer: `relative` positions an element relative to its own normal position, without removing it from document flow. `absolute` removes it from flow and positions it relative to the nearest positioned ancestor (or the viewport if none exists). `fixed` positions relative to the viewport and stays put during scroll. `sticky` behaves like `relative` until a scroll threshold is crossed, then behaves like `fixed` within its containing block — commonly used for sticky headers/table headers.

SECTION C: JAVASCRIPT — FUNDAMENTALS TO ADVANCED

Q11: Explain hoisting.

Answer: Before code executes, the JS engine scans for variable and function declarations and allocates memory for them — this is hoisting. `var` declarations are hoisted and initialized to `undefined`. `let`/`const` are hoisted but not initialized — accessing them before their declaration line throws a `ReferenceError` (the "temporal dead zone"). Function declarations (`function foo(){}`) are fully hoisted, body included, so they can be called before their line in the source. Function expressions and arrow functions assigned to `let`/`const`/`var` are not.

js

```
console.log(a); // undefined
var a = 5;

console.log(b); // ReferenceError
let b = 5;
```

Q12: Explain prototypal inheritance.

Answer: JS objects have an internal link (`[[Prototype]]`), accessible via `__proto__` or `Object.getPrototypeOf`) to another object. When you access a property that doesn't exist on the object itself, JS walks up this prototype chain until it finds it or reaches `null`. `class` syntax in modern JS is syntactic sugar over this same mechanism — it doesn't introduce classical inheritance, just a cleaner syntax for prototype-based inheritance.

```
js

function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return `${this.name} makes a sound`; };

const dog = new Animal('Rex');
dog.speak(); // 'Rex makes a sound' - found via prototype chain
```

Q13: What's the difference between `==` and `===`?

Answer: `===` (strict equality) compares value *and* type with no coercion. `==` (loose equality) coerces types before comparing, which leads to famously confusing results (`'' == 0` is `true`, `null == undefined` is `true` but `null === undefined` is `false`). Default to `===` always; the only common accepted exception is `== null` as a shorthand to check for both `null` and `undefined` at once.

Q14: Explain `call`, `apply`, and `bind`.

Answer: All three let you explicitly set what `this` refers to inside a function. `call(thisArg, arg1, arg2, ...)` invokes immediately with args listed individually. `apply(thisArg, [argsArray])` invokes immediately with args as an array. `bind(thisArg, ...)` doesn't invoke — it returns a *new function* permanently bound to that `this`, useful for event handlers or passing a method as a callback without losing context.

```
js
```

```
const user = { name: 'Sam' };
function greet(greeting) { return `${greeting}, ${this.name}`; }

greet.call(user, 'Hi');           // 'Hi, Sam'
greet.apply(user, ['Hi']);        // 'Hi, Sam'
const bound = greet.bind(user);
bound('Hi');                     // 'Hi, Sam'
```

Q15: What's the difference between deep and shallow copy? How do you deep clone an object in JS?

Answer: A shallow copy (`{...obj}`, `Object.assign`, `Array.slice`) copies top-level properties, but nested objects/arrays are still shared references — mutating a nested value affects both copies. A deep copy recursively duplicates every level. Modern option: `structuredClone(obj)` (native, handles most cases including circular refs). Older approach: `JSON.parse(JSON.stringify(obj))` — works for plain data but silently drops functions, `undefined`, `Date` becomes a string, etc.

Common mistake: Assuming `{...obj}` is a safe deep copy — this is one of the most common React state-mutation bugs (mutating nested state and wondering why re-render doesn't happen).

Q16: Explain `Promise` states and write a function that retries a failed fetch up to N times.

Answer: A Promise is pending → then settles as either fulfilled or rejected, exactly once (settling is final — a promise can't change state twice).

```
js
async function fetchWithRetry(url, retries = 3, delay = 500) {
  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return await res.json();
    } catch (err) {
      if (attempt === retries) throw err;
      await new Promise(r => setTimeout(r, delay * (attempt + 1))); // simple b
    }
  }
}
```

Why asked: Tests real async control flow under a realistic constraint, not just "do you know what `await` does."

Q17: What is currying? Write a curried `add` function.

Answer: Currying transforms a function taking multiple arguments into a sequence of functions each taking one argument, returning a new function until all arguments are collected.

```
js
const add = a => b => c => a + b + c;
add(1)(2)(3); // 6
```

Why asked: Less common in daily work but tests comfort with closures and functional patterns — sometimes shows up in a "write a `pipe`/`compose` function" variant too.

Q18: Explain `null` vs `undefined`.

Answer: `undefined` means a variable has been declared but not assigned a value, or a function returned nothing, or an object property doesn't exist. `null` is an intentional, explicit "no value" assignment by the programmer. `typeof undefined === 'undefined'`; `typeof null === 'object'` (a long-standing JS quirk/bug preserved for backwards compatibility).

Q19: What is the difference between synchronous and asynchronous code, and how does `async/await` relate to Promises?

Answer: Synchronous code runs top to bottom, blocking further execution until each line finishes. Async code allows long-running operations (network requests, timers) to run without blocking the main thread — the rest of the code continues, and a callback/Promise resolves later. `async/await` is syntactic sugar over Promises: an `async` function always returns a Promise, and `await` pauses execution *within that function* (not the whole program) until the awaited Promise settles, letting you write async code that reads like synchronous code.

Q20: Live-coding staple: implement `debounce` AND `throttle`, and explain the difference out loud.

```
js
```



```

function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

function throttle(fn, limit) {
  let inThrottle = false;
  return (...args) => {
    if (!inThrottle) {
      fn(...args);
      inThrottle = true;
      setTimeout(() => { inThrottle = false; }, limit);
    }
  };
}

```

Explain the difference clearly: Debounce waits for a pause in activity before firing (good for search-as-you-type). Throttle guarantees the function fires at most once per interval regardless of how often it's called (good for scroll/resize handlers where you want regular updates, not a single delayed one).

Rapid-Fire Round (know these cold, one-sentence answers)

- **Q: What's an IIFE?** A: Immediately Invoked Function Expression — a function defined and executed at once, historically used to create private scope before ES modules existed. `(function(){ ... })();`
- **Q: What's the difference between `Array.forEach` and `Array.map`?** A: `forEach` returns `undefined` and is for side effects; `map` returns a new array and is for transformation.
- **Q: What's a pure function?** A: Same input always produces same output, no side effects, doesn't mutate external state.
- **Q: What's the difference between `Object.freeze` and `const`?** A: `const` prevents reassignment of the binding; `Object.freeze` prevents mutation of the object's own properties (shallow only).
- **Q: What does `NaN === NaN` evaluate to, and why?** A: `false` — NaN is never equal to anything, including itself, per the IEEE 754 spec. Use `Number.isNaN()` to check.
- **Q: What is destructuring?** A: Syntax to unpack values from arrays/objects into variables: `const { name, age } = user;`

- **Q: What's optional chaining and nullish coalescing?** A: `user?.address?.city` safely accesses nested properties without throwing if intermediate values are null/undefined; `value ?? 'default'` falls back only on `null`/`undefined` (unlike `||`, which also falls back on falsy values like `0` or `''`).
-

End of Part 1. Say "next part" for Part 2: TypeScript & React Fundamentals.